FLYRANK.AI · AI BACKEND ENGINEERING · PROJECT BRIEF

Auth · Login & protect

Build a secure API with Supabase Auth — sign up, log in, log out — verify JSON Web Tokens, guard protected routes, document it in Swagger UI, and publish to GitHub.

~6–8 h across 6 stages | Stretch +2 h | Bonus AI stage +1 h | JavaScript or Python | $0 · no credit card

COMPANION DECK
Who are you? Authentication & web security

YOU WILL PRACTICE
Supabase Auth · JWT verification · protected routes · auth middleware · Swagger bearer auth

DELIVERABLE
Public GitHub repo, ≥6 commits, README

How to read this document: new words are shown in **bold** the first time they appear — every one is explained in the Glossary at the end. Work the stages **in order**; each ends with a checkpoint that proves it works. **Every underlined green word is a clickable link** — a doc, tool, or tutorial — collected by stage in the Curated resources section.

Contents

1. Goal & purpose 1
2. The big idea in 60 seconds 2
3. Tools — pick one lane 2
4. The task — six stages 3
5. Bonus stage — the AI rematch 7
6. Requirements & stretch goals 8
7. Done means 8
8. Curated resources 9
9. Glossary 12

1 · Goal & purpose

Goal: Build a secure **API** that handles user **authentication** — Sign Up, Log In, Log Out — and **protects** specific routes so they answer only for logged-in users. You will use **Supabase Auth** as your **Identity Provider** to manage accounts and issue **JSON Web Tokens (JWTs)**, verify those tokens to guard "user-only" endpoints, document the flow in **Swagger UI**, and publish everything to **GitHub**.

In the previous projects your API was wide open: anyone who knew the URL could read, create, or delete data. That's fine for a to-do demo — and a disaster for anything real. A social network only lets you edit *your* posts; a shop only shows *your* cart; FlyRank only lets authenticated users see *their* SEO audits. This project turns the auth fundamentals into muscle memory: your backend learns *who is calling* before it decides what to allow.

One deliberate choice runs through the whole task: **you do not write cryptography or password-hashing yourself.** Rolling your own auth is how careers end — in the real world you lean on a trusted Identity Provider. Supabase stores the accounts, hashes the passwords, and signs the tokens; your job is the part that matters for a backend developer: **receiving a token, verifying it, and opening (or refusing) the door.**

> Beginners usually overthink this. There's no maths and no magic — the entire secured API is a few small routes plus one **middleware** function, built in stages where you never write more than ~15 lines before you can test again. And it produces the "current user" that the upcoming tenant-isolation project, and every capstone, depends on.

2 · The big idea in 60 seconds

Secure authentication is a **trust triangle** between three parties: the **client**, your **backend server**, and the **Identity Provider** (Supabase). Nobody trusts a plain password sitting on your server — instead, Supabase vouches for the user by signing a token, and your server checks that signature.

Step	Who does it	What happens
1. Sign up / Log in	Client → Supabase	The client sends **credentials** (email + password) to Supabase.
2. The token	Supabase → Client	Supabase checks the credentials and returns a **JWT** (the **access token**).
3. The request	Client → your server	The client calls your backend, attaching the JWT in an **Authorization header**.
4. Verification	Your server → Supabase	Your server asks Supabase "is this token real?" If yes, the **protected** door opens.

That table **is** the project. Everything below just builds it, one row at a time. Here are the five doors you'll put in the wall:

Route	Purpose	Auth header
`POST /auth/signup`	Create a new user account	none
`POST /auth/login`	Authenticate & return a JWT	none
`POST /auth/logout`	End the user's session	`Authorization: Bearer <token>`
`GET /protected/profile`	Read private profile data	`Authorization: Bearer <token>`
`GET /public/info`	Read public, open data	none

3 · Tools — pick ONE lane

Both lanes build the same secure API. Pick the language you've used since Task 1 and stay in it — this is the same repo growing. Every row is free, no credit card.

	JavaScript lane	Python lane
Language	Node.js (free, nodejs.org)	Python 3.10+ (free, python.org)
Framework	Express — reuse your earlier app · middleware guide	FastAPI — security tutorial
Identity Provider	Supabase — free project, no card	same
Auth SDK	`@supabase/supabase-js`	`supabase` (PyPI)
Config / secrets	`.env` (`node --env-file` or `dotenv`)	`.env` (`python-dotenv`)
Swagger UI	Add with swagger-ui-express (Stage 5)	**Built in** at `/docs` — zero setup

Testing your API	curl + browser + Hoppscotch	same
Publishing	Git + a free GitHub account	same

> Not sure? If you liked the JS 101 session, take the **JavaScript** lane and add auth to the Express app you already have. If Python feels friendlier, take the **Python** lane — FastAPI gives you the Swagger "Authorize" padlock almost for free. **Don't switch lanes mid-project.**

4 · The task — six stages (+ one bonus)

Work stage by stage, **in order**. Each stage ends with a checkpoint: a command you run to prove it works. Commit to Git after every stage (that's your ≥6 commits, honestly earned). **If you only finish Stage 3, ship it anyway — a working half is worth more than a broken whole.**

Golden rule for this brief: **you never store a password and you never hash anything yourself.** Supabase does that. Your code only ever *sends* credentials to Supabase and *verifies* the tokens it hands back.

0 Set up Supabase & your server

~45 min

Before you can guard a castle, you build the guard tower.

1. Create a free account at supabase.com and spin up a new project (call it anything, e.g. `Auth-Practice`). It takes a minute or two to provision.
2. In your **Supabase Dashboard**, open **Project Settings → API** and copy two values: your **Project URL** and your **anon key** (the public key — safe to use from your app). *Never* use the `service_role` key here; it bypasses all security.
3. Initialize your project locally and create a git-ignored `.env` for these secrets:

```
SUPABASE_URL=your_project_url
SUPABASE_KEY=your_anon_key
PORT=3000
```

4. Install your dependencies (framework + the **Supabase SDK** + a dotenv loader) and **initialize the Supabase client** from those environment variables. Start the server on your port (Express `3000`, FastAPI `8000`).

> **One-time setting that saves you an hour:** by default Supabase makes new users confirm their email before they can log in. For this practice project, open **Authentication → Sign In / Providers → Email** and turn **"Confirm email" off**, so a fresh signup can log in immediately. (In production you'd leave it on — email confirmation is a real security feature.)

> **CHECKPOINT** — run your server. It logs something like `Server running and connected to Supabase` and throws no errors. Your `.env` is already in `.gitignore`.

Commit: `Stage 0: setup server and supabase client`

1 Open auth: Sign Up & Log In

~1 h

The front gates open. Let users register their keys and come back with them.

1. Create `POST /auth/signup`. It expects a **JSON** body with `email` and `password`.
   - Call the sign-up method — **JS** `supabase.auth.signUp()` · **PY** `supabase.auth.sign_up()`.
   - **Validate**: if `email` or `password` is missing, return **400** ("Bad Request") with a JSON error — the server never trusts the client.
   - On success, return **201** ("Created") with the user object Supabase returns.
2. Create `POST /auth/login`.
   - Call the sign-in method — **JS** `signInWithPassword()` · **PY** `sign_in_with_password()`.
   - **Validate** empty fields → **400**. If Supabase rejects the credentials, return **401** with `{ "error": "Invalid login credentials" }`.
   - On success, return **200** with the **access token** (JWT) and **refresh token** Supabase provides.

CHECKPOINT — register, then log in:

```
curl -i -X POST http://localhost:3000/auth/signup \
  -H "Content-Type: application/json" \
  -d '{"email":"test@example.com","password":"password123"}'
```

returns `201`. Then hit `/auth/login` and confirm you get an `"access_token"` string back. Signup with no password → `400`.

Commit: `Stage 1: signup and login routes working`

2 The public & protected gates

~1 h

Build a public lobby anyone can enter, and a locked door — even before you've hired the guard.

1. Add a **public endpoint** `GET /public/info`. No auth. Returns `{ "message": "Welcome stranger! This info is public." }` with status **200**.
2. Add a **protected endpoint** `GET /protected/profile`. To reach it, a client must send their access token in the **Authorization header** as `Authorization: Bearer <token>`.
   - Inside the route, **extract** the token from that header.
   - If the header is missing, malformed, or has no token, immediately return **401** with `{ "error": "Access token required" }`. (You're not verifying the token *yet* — just checking one was presented.)

CHECKPOINT

```
curl -i http://localhost:3000/public/info        # -> 200
curl -i http://localhost:3000/protected/profile  # -> 401 (no token sent)
```

Commit: `Stage 2: public route and unverified protected route`

3 The guard: token verification

~1 h

The guard at the door inspects the visitor's pass — and turns away forgeries.

Now make `GET /protected/profile` actually **verify** the token with Supabase.

1. Extract the token from the header (as in Stage 2).
2. Ask Supabase whether it's real — **JS** `supabase.auth.getUser(token)` · **PY** `supabase.auth.get_user(token)`. This makes a network call to Supabase, so the answer is trustworthy.
3. If the token is expired, tampered with, or invalid → return **401** with `{ "error": "Invalid or expired token" }`.
4. If it verifies → return **200** with the user's safe metadata (id, email, account-created date).

> **CHECKPOINT** — log in, copy the `access_token`, and pass it:
>
> ```
> curl -i http://localhost:3000/protected/profile \
>   -H "Authorization: Bearer <PASTE_YOUR_ACCESS_TOKEN_HERE>"
> ```
>
> returns `200` and your user details. Now **change one character** of the token and run again → `401`. You just watched a forged pass get rejected.

Commit: `Stage 3: profile route token verification`

4 Middleware protection & logout

~1.5 h

One guard, standing at every locked door — instead of the same check pasted into each room.

1. **Middleware guard.** Copy-pasting token-checking into every protected route gets messy and dangerous (miss one, and a door is unguarded). Extract the Stage 3 logic into a reusable **middleware** function (Express) or **dependency** (FastAPI). Apply it to `GET /protected/profile` so the route body only runs *after* the guard verifies the user and attaches it (think `req.user` / the injected user).
2. Create `POST /auth/logout` — a **protected** route (it uses your new guard). Call the sign-out method — **JS** `supabase.auth.signOut()` · **PY** `supabase.auth.sign_out()`. Return **204** ("No Content") on success.
3. **Stop and notice:** you have built a complete authentication system — sign up, log in, log out, and a single reusable guard protecting any route you point it at.

> **CHECKPOINT** — add a second protected route, e.g. `GET /protected/dashboard`, using the *same* middleware — write no new auth code. Confirm it rejects a bad token with `401` and permits a valid one with `200`. That reuse is the whole point.

Commit: `Stage 4: auth middleware and logout endpoint`

5 See it: Swagger UI

~1–1.5 h

So far you've imagined the locked doors. Now put visible padlocks on them.

Swagger UI turns your API into an interactive page with a **Try it out** button — and a lock icon that lets you paste a token once and reuse it. You just have to *tell* it which doors are locked.

- **Python lane:** FastAPI already serves Swagger at `http://localhost:8000/docs`. To make the **Authorize** padlock appear, configure FastAPI's `HTTPBearer` security scheme and apply it to your protected routes.
- **JavaScript lane:** in your `openapi.json`, define a `securitySchemes` block of `type: http`, `scheme: bearer`, `bearerFormat: JWT`, and link it to your `/protected/` and `/auth/logout` endpoints. Serve it with swagger-ui-express at `/docs`.

Then, in the browser: click **Authorize**, paste a JWT from your login, and run **Try it out** on `GET /protected/profile` — no curl needed.

> **CHECKPOINT** — `/docs` shows a **lock icon** next to your protected routes; you can authorize with a token and successfully call the profile endpoint from the browser. Take a **screenshot** of Swagger for the README.

Commit: `Stage 5: Swagger UI documentation with bearer auth`

6 Publish to GitHub

~1 h

Never share your secrets. Always share your code.

1. Create a **public GitHub repo** and push your project (your ≥6 stage commits along with it).
2. **CRITICAL:** confirm your `.env` is listed in `.gitignore` and was never committed. Your Supabase keys must not reach GitHub — bots scrape new keys within a minute. Commit a `.env.example` with the key names and placeholder values instead.
3. Write a professional **README** covering: what the project is; how to set up the environment variables; the one command to run it; an **API reference table** of the five endpoints and whether each needs auth; and your **Swagger screenshot**.

> **New to Git?** `init → add → commit → push` is all you need here — see the Git & GitHub guide in your vault.

> **CHECKPOINT** — a peer can clone your repo, drop in their own `.env` values, and run your authenticated API in under 5 minutes.

Commit: `Stage 6: publish to GitHub and write README` — then push everything.

★ Make it yours — optional extras

optional · as many or as few as you like

You have real user accounts now — do something with them.

None of these are required. Pick whatever sounds fun (creative alternatives welcome):

- **Read the token yourself:** paste an access token into jwt.io and look at the decoded **payload**. Write two sentences on what's inside a JWT — and why you never put a secret in one (anyone can read it).
- **The expiry experiment:** log in, wait for the access token to expire (Supabase default: one hour), then call `/protected/profile` with the stale token. Note the `401`. *This is the entire reason **refresh tokens** exist.*
- **A `403` case:** add a route a logged-in user can reach but only an "admin" may use; return **403** ("Forbidden") for everyone else. `401` = "I don't know you"; `403` = "I know you, and no." That difference is **authorization**.
- **A real logout test:** log out, then reuse the same token on a protected route and record what happens — why stateless JWTs make "instant logout" genuinely hard.
- **Refresh flow:** use the **refresh token** from login to obtain a new access token without logging in again.

Commit (if you build any): `Extras: <what you added>`

5 · Bonus stage — the AI rematch

7 The AI rematch

~1 h · optional — and the most fun

You secured an API by hand, door by door. Now hire the fastest junior developer on Earth to do it — and review their work.

You did Stages 0–6 by hand for a reason: you now know exactly what a correct, secure auth flow looks like. That knowledge turns this stage from a magic show into a **code review**.

1. **Write the prompt yourself — this is the real exercise.** Without copying from this document, write your own **prompt** asking an AI to build the same secured API. From memory, specify: your lane and framework, Supabase as the Identity Provider, the five routes, the status codes (`201` / `200` / `204` / `400` / `401`), token verification via **middleware**, and the Swagger bearer setup.
2. **Generate in quarantine.** Put the AI's code in a separate folder (`ai-version/`) or branch. Your Stages 0–6 code stays untouched — that is your deliverable.
3. **Run it.** Does it start? Fire your Stage 3 and Stage 4 checkpoints at it — a valid token, then a tampered one.
4. **Diff it.** Compare with yours (`git diff --no-index your-file ai-file`), then answer three questions in an **"AI vs me"** README section:
    - How did it handle **token extraction** — did it correctly parse the `"Bearer "` prefix, or would `Authorization: <token>` slip through or crash?
    - What **security flaws** might it have introduced — does it safely reject an invalid token, or trust `getUser` without checking the error? Did it leak the `service_role` key or log the token?
    - What did *your prompt* forget to specify — and what did the AI silently decide for you?
5. **One rematch.** Improve your prompt with what you learned, regenerate, and note in one sentence what changed.

> The lesson hiding in this stage: an AI's output is exactly as good as your specification — and you could only judge it because you built the thing yourself first. **Both halves of that sentence are your career from now on.**

> **CHECKPOINT** — your README has an "AI vs me" section with your full prompt and ≥3 concrete differences.

Commit: `Stage 7: AI vs me` (AI code stays in its own folder/branch).

6 · Requirements

Done = every box ticked. Each one is checkable in under a minute.

- [ ] Server starts on localhost with a **single documented command**, and connects to Supabase without errors.
- [ ] `.env` **is used and git-ignored**; `.gitignore` prevents it from being pushed, and a `.env.example` is committed. No Supabase keys anywhere in git history.
- [ ] `POST /auth/signup` and `POST /auth/login` talk to **Supabase Auth** successfully; login returns an **access token**.
- [ ] `GET /protected/profile` **extracts and verifies** the bearer token from the Authorization header via Supabase.
- [ ] Correct **status codes**: `201` signup · `200` login/read · `204` logout · `400` missing input · `401` on missing, malformed, or invalid/expired token — each error with a JSON `error` message.
- [ ] The auth check is a **reusable middleware/dependency**, applied to more than one protected route.
- [ ] **Swagger UI at** `/docs` shows the lock icon on protected routes, and Bearer authorization works end-to-end from the browser.
- [ ] Public **GitHub repo** with **≥6 clean commits** and a comprehensive README: setup, run command, endpoint/auth table, and the Swagger screenshot.

Stretch (optional)

- [ ] Add a real `403` ("Forbidden") case — an authenticated user who isn't allowed — and explain the `401`-vs-`403` difference in your README.
- [ ] Add a **refresh-token** endpoint that exchanges a refresh token for a fresh access token; write one line on why access tokens are short-lived.
- [ ] **Rate-limit** `POST /login` and return **429** after N failed attempts — then explain why brute-force protection lives here.
- [ ] Stage 7 — the AI rematch (see above): prompt an AI to build the same secured API, run it, diff it, write your "AI vs me" section.

7 · Done means

- The full flow works twice: once via `curl -i` (signup → login → protected call succeeds → tampered token gives `401`), and once via **Swagger UI** "Authorize" + "Try it out".
- Your repo is public, `.env` appears **nowhere** in git history, and `git log` shows one honest commit per stage.

8 · Curated resources

Don't read everything. Each section says *when* you need it. All resources are free, no credit card. Every underlined title is a clickable link.

● **Start here** (everyone) · ● **When you're comfortable** · ● **Optional deep dive** · **JS** / **PY** mark lane-specific rows.

§1 · Before you start — what auth & JWTs are

	Resource	Format	Why it's useful
●	JWT — Introduction (jwt.io)	Article, ~8 min	The clearest short explainer of a JWT and its three parts. The "access token" stops being mysterious.
●	Supabase — Auth overview	Official, ~10 min	What an Identity Provider does for you: accounts, hashed passwords, signed tokens. The big-picture page.
●	Auth0 — JWT structure	Article, ~10 min	A deeper look at what's inside a token — useful for the "read the token yourself" extra.
●	Supabase — JSON Web Tokens	Official, ~12 min	Exactly how Supabase signs and verifies tokens — read when curious *how* `getUser` can trust a token.

§2 · Supabase project & secrets — Stage 0

	Resource	Format	Why it's useful
●	Supabase — create a free project	Web app	Sign up (no card), create a project, find your Project URL and anon key under Settings → API. Stage 0.
●	Internal — Env vars & secrets guide	Internal guide	How to put your Supabase keys in a git-ignored `.env`, read them, and never leak them. Read before your first commit.
●	Supabase — anon vs service_role key	Official	Which key is safe where: the anon key is fine in your app; the `service_role` key must stay secret.

§3 · The Supabase SDK — the exact auth calls (Stages 1, 3, 4)

	Resource	Format	Why it's useful
●	JS `signUp()` · `signInWithPassword()`	Official reference	The register + login calls for the JS lane, with copy-ready examples. Stage 1.
●	JS `getUser(jwt)` · `signOut()`	Official reference	Verify a token (Stage 3) and end a session (Stage 4). `getUser` makes a real network call.
●	PY `sign_up()` · `sign_in_with_password()`	Official reference	The Python-lane register + login calls, same shape as JS. Stage 1.
●	PY `get_user(jwt)`	Official reference	Verify a token in the Python lane. Stage 3–4.

§4 · Build-along tutorials

Watching or typing along is allowed and encouraged — just make the final code yours (your routes, your validation, your extras).

	Resource	Format	Why it's useful
●	PY Integrating FastAPI with Supabase Auth	Article, ~20 min	The closest worked example for Python: signup/login/signout plus a bearer dependency — almost exactly Stages 1–4.
●	Supabase for Beginners (freeCodeCamp)	Article, ~25 min	A calm, language-agnostic walk through what Supabase is and how its client and auth work. Read before Stage 1 if the SDK feels new.
●	Introduction to Supabase — full tutorial (freeCodeCamp)	Video, ~1 h	A free end-to-end video if you learn by watching. The auth SDK calls are identical in plain Express/FastAPI — take the calls, ignore the front-end.

§5 · Authorization header & status codes — Stages 2–4

	Resource	Format	Why it's useful
●	MDN — Authorization header	Official reference	The exact shape of `Authorization: Bearer <token>` you extract in Stage 2.
●	MDN — 401 · 403	Official reference	When to send which — and why `401` ("who are you?") and `403` ("no.") are different answers.
●	JS Express — Writing / Using middleware	Official guide	How to write the reusable guard for Stage 4 and apply it to a route.
●	PY FastAPI — Security & dependencies	Official tutorial	FastAPI's `Depends(...)` is your "middleware" — the bearer-token dependency pattern Stage 4 needs.
●	http.cat	Reference (fun)	A cat per status code. A surprisingly sticky way to remember `401` vs `403`.

§6 · Swagger UI with bearer auth — Stage 5

	Resource	Format	Why it's useful
●	PY FastAPI — HTTPBearer	Official reference	Configure the `HTTPBearer` scheme so the Authorize padlock shows up on `/docs` automatically.
●	JS Swagger — Bearer authentication	Official docs	The exact `securitySchemes` block to put in your `openapi.json`.
●	JS swagger-ui-express (npm)	Package docs	Serve your `openapi.json` at `/docs`, the same tool you used in Task 1.
●	PY FastAPI — First steps (auto `/docs`)	Official tutorial	Reminder that FastAPI generates Swagger for free — you only add the security scheme.

§7 · Publish & secure — Stages 0 & 6

	Resource	Format	Why it's useful
●	Internal — Git & GitHub guide	Internal guide	`add → commit → push`, plus `.gitignore` so `.env` never leaves your machine. Everything Stage 6 needs.
●	GitHub Docs — Hello World	Official, ~15 min	The fastest refresher on repos, commits, and pushing if your Task 1 setup is hazy.

🟡	OWASP — Authentication Cheat Sheet	Reference	The professional checklist for auth. Skim after Stage 4 to see which rules Supabase already handles for you.
🟠	OWASP Top 10 · jwt.io decoder	Reference / tool	The canonical web-security risk list, and a tool to decode your own token. Both optional deep dives.

Suggested path for a complete beginner

1. Before Stage 0: §1 (both green reads) — ~15 min, so the trust triangle makes sense.
2. Stage 0: create the Supabase project (§2) and set up your `.env`.
3. Stages 1–4: keep your lane's SDK pages (§3) and one build-along (§4) open; build a stage, test, repeat. Reach for the header/status references (§5) when the guard misbehaves.
4. Stage 5: your lane's Swagger row (§6). Stage 6: the internal Git guide (§7).
5. Curious or stuck: the yellow rows. Everything red is optional forever.

9 · Glossary

Plain-language definitions of every **bold** word above. No definition depends on another — read them in any order.

Word	What it means
API	The set of doors your program offers so other programs can talk to it — here, the five auth/profile endpoints.
Authentication	Verifying *who* a user is (matching an email and password). The question `POST /login` answers.
Authorization	Verifying *what* an already-known user is allowed to do. The question a `403` answers.
Identity Provider (IdP)	An external service — Supabase here, or Auth0, Firebase, Google — that manages accounts, passwords, and tokens so your server never has to.
Supabase Auth	The authentication part of Supabase: it stores users, hashes passwords, and issues signed JWTs. You call it; you don't reimplement it.
Credentials	The email and password a user proves themselves with. Your server forwards them to Supabase and never stores them.
JSON Web Token (JWT)	A compact, signed string that carries claims (like "this is user 123"). Because it's cryptographically signed, it can't be secretly altered — change one character and verification fails.
Access token	The specific JWT a client sends on each request to prove it's logged in. Short-lived (Supabase's default is one hour).
Refresh token	A longer-lived token used to get a *new* access token when the old one expires, without making the user log in again.
Bearer token	Any token the server honours for whoever "bears" (presents) it, sent as `Authorization: Bearer <token>`. Your access token is used as a bearer token.
Authorization header	The standard HTTP header a client uses to send a token: `Authorization: Bearer eyJhbGciOi...`.
Protected / protect	A route that runs its logic only after a valid token is verified; without one it answers `401`.
Middleware	A function that runs *before* your route handler and can stop the request early. Your auth guard is middleware: it verifies the token, attaches the user, then lets the route run.
Dependency (FastAPI)	FastAPI's version of middleware for a route: a function FastAPI runs first (to verify the token) and injects its result into your handler.
Extract (a token)	To pull the token string out of the `Authorization: Bearer <token>` header before verifying it.
Validate / validation	Checking incoming data before trusting it — here, that `email` and `password` are present — and rejecting bad input with `400`.
Status code	The 3-digit number in every response saying how it went: `200` OK · `201` Created · `204` No Content · `400` Bad Request · `401` Unauthorized · `403` Forbidden · `429` Too Many Requests.
401 vs 403	`401` Unauthorized = "I don't know who you are" (missing/bad token). `403` Forbidden = "I know exactly who you are, and you still may not."
Environment variable / `.env`	A configuration value handed to your program at startup. Secrets like your Supabase keys live in a git-ignored `.env` file, never in code.

`.env.example`	A committed copy of `.env` with the key names and placeholder values, so a teammate knows what to set without seeing your real secrets.
anon key	Supabase's public API key, safe to use from your app. Distinct from the `service_role` key, which bypasses security and must stay server-side and secret.
Supabase Dashboard	Supabase's web console where you find your Project URL and keys and change auth settings (like turning off email confirmation).
Swagger UI	A web page that renders your API as interactive docs with a "Try it out" button and, once configured, an "Authorize" padlock for pasting a token.
securitySchemes / HTTPBearer	The OpenAPI/FastAPI configuration that tells Swagger a route is locked behind a bearer token — that's what makes the padlock appear.
curl	A terminal program that sends HTTP requests. `curl -i URL` shows the status code and headers, so you can see a `201` or `401` directly.
Git / GitHub / repo / commit	Git tracks versions of your code in a **repo**sitory; a **commit** is one saved step with a message; GitHub hosts the repo online so others can clone and run it.
README	The front page of a repo: what the project is, how to configure it, and how to run it.
Prompt	The instructions you give an AI assistant. In Stage 7 your prompt is a mini-specification of the whole secured API — the more precise it is, the closer the output lands to yours.
Diff	A line-by-line comparison of two versions of code showing what changed. `git diff` produces one; reading diffs is how professionals review each other's work.

FlyRank.ai · AI Backend Engineering · Project Brief — Auth · Login & protect. All linked resources are free with no credit card required.